

Software Requirements Specification 2: BookForMe

Muhammad Ahmad Humayun Hanif, Jazib Waqas, Taha Hunaid Ali, Muhammad Taqi

November 8, 2025

Contents

1	System Block Diagram	2
1.1	System Block Diagram	2
1.2	Description of Components	2
2	Project Plan	3
2.1	Objectives	3
2.2	Monthly Timeline	4
2.3	Team Roles and Responsibilities	4
2.4	Risks and Mitigation Strategies	4
3	Wireframes	6
3.1	Overview	6
3.2	User Interface Mockups	6
3.3	Vendor Interface Mockups	6
3.4	Risks and Mitigation Strategies	6

1 System Block Diagram

1.1 System Block Diagram

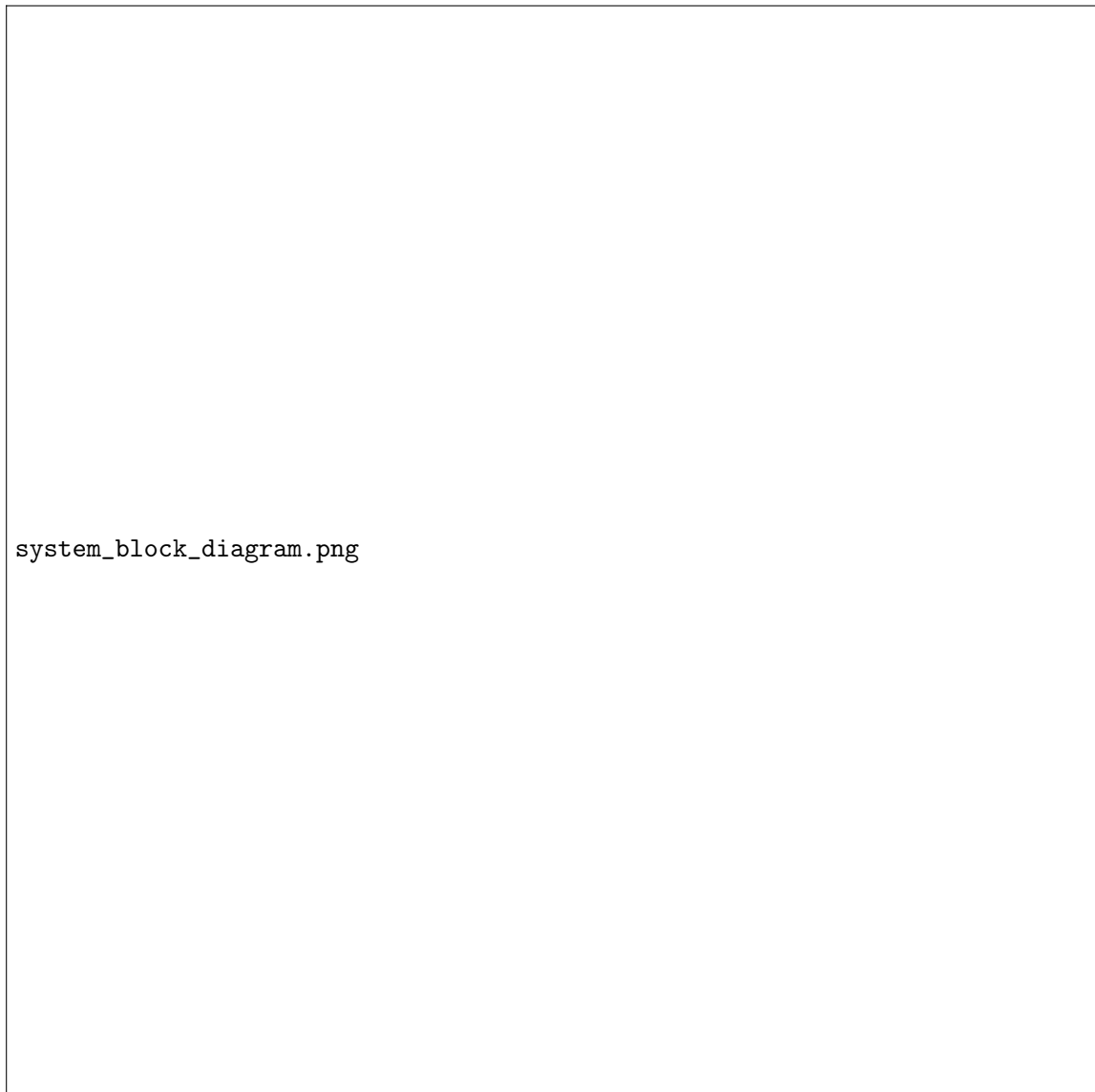


Figure 1: System Block Diagram for BookForMe Platform

1.2 Description of Components

The BookForMe system follows a modular, service-oriented architecture composed of three primary layers: **Frontend Layer**, **Backend/API Layer**, and **External Integration Layer**.

- **User Web Application (Frontend)**

Developed using React.js (or Next.js), this component allows users to browse, book, and interact with vendors and other users. It communicates with the backend through RESTful or GraphQL APIs hosted on Firebase Cloud Functions.

- **Vendor Dashboard**

A secure administrative interface where vendors can manage bookings, view calendar integrations, approve payments, and link third-party accounts such as WhatsApp Business and Google Sheets.

- **AI Agent (Backend)**

A Node.js or Python-based service that connects to the WhatsApp Business API, processes incoming messages, and interprets intents using a Natural Language Understanding (NLU) pipeline. It interacts with the Firestore database for state management and booking synchronization.

- **Cloud Firestore Database**

Acts as the central data repository for users, vendors, bookings, messages, and system logs. Firestore's real-time capabilities enable instant updates across devices and platforms.

- **Google Sheets API Integration**

Used by vendors who maintain availability records externally. The AI Agent synchronizes this data, ensuring consistency with the Firestore booking system.

- **WhatsApp Business API**

Enables two-way communication between vendors and customers. The AI Agent receives webhooks from WhatsApp and replies contextually, managing availability queries and booking confirmations.

- **Authentication and Security Layer**

Firebase Authentication and HTTPS/TLS encryption are employed to ensure secure data transactions between all components. OAuth 2.0 is used for vendor integrations.

Overall, the architecture enforces modularity, concurrency control (via Firestore Transactions), and horizontal scalability via cloud deployment on Firebase or Google Cloud Functions.

2 Project Plan

2.1 Objectives

The overarching objectives of the BookForMe project are to:

1. Develop a centralized booking platform connecting users and vendors across informal service domains.
2. Implement an intelligent AI Receptionist capable of processing WhatsApp-based booking requests using Natural Language Understanding (NLU).

3. Design a scalable, secure backend integrated with Firebase, Google Sheets API, and WhatsApp Business API.
4. Deliver a seamless, user-friendly frontend and vendor dashboard ensuring accessibility, reliability, and real-time synchronization.

2.2 Monthly Timeline

2.3 Team Roles and Responsibilities

- **M. Ahmad Hanif (Project Lead)** — Leads system architecture and AI Receptionist integration with Firestore and Google Sheets. Responsible for development and functioning of the NLU pipeline. Oversees project roadmap, documentation, and delivery milestones.
- **Jazib Waqas (Backend Engineer)** — Responsible for backend architecture, database integration, and implementation of booking, vendor, and authentication APIs. Leads WhatsApp API integration and ensures secure communication between the conversational agent and backend services.
- **Taha Hunaid Ali (Frontend Developer)** — Develops and maintains user-facing and vendor dashboards using React.js and Tailwind CSS. Works on real-time interface features such as chat, leaderboard, and vendor analytics visualization. Coordinates frontend-backend linkage and deployment readiness.
- **Muhammad Taqi (AI Engineer)** — Works on the design and development of the NLU pipeline, focusing on intent detection, slot extraction, and model fine-tuning. Collaborates on AI Receptionist integration and ensures accurate handling of bilingual booking queries.

2.4 Risks and Mitigation Strategies

- **Data Quality Risk:** Incomplete or noisy bilingual data may reduce model accuracy.
Mitigation: Continuous dataset refinement and augmentation using backtranslation and synonym replacement.
- **Integration Challenges:** WhatsApp or Google APIs may change or limit functionality.
Mitigation: Maintain modular code with fallback REST APIs and mock endpoints.
- **Performance Risk:** AI model latency during live booking.
Mitigation: Optimize model inference and use caching for repetitive queries.

Month	Tasks	Deliverables
November: Foundation Phase	• Finalize <i>SRS 1</i>, <i>SRS 2</i> & <i>SDS</i> and complete the literature review. • Build an initial dataset for booking requests and vendor replies. • Design and implement a low-fidelity UI prototype for early validation. • Set up GitHub repositories and development environments.	• Approved SRS Document. • Preliminary Bilingual Dataset. • Prototype User Interface (Login, Vendor Discovery).
December: Baseline Model and Backend Setup	• Train baseline NLU models for intent detection and slot extraction. • Implement backend integrations using Firestore.	• Baseline NLU Model. • Defined Database Schema. • Minimal Working Booking System (Storage & Retrieval).
January: AI Receptionist and WhatsApp Integration	• Fine-tune the NLU pipeline for improved bilingual performance. • Implement the AI Receptionist for WhatsApp-based booking processing. • Introduce concurrency control, safety checks, and backend error handling. • Refine frontend booking and profile management interfaces.	• Functional AI Receptionist for End-to-End Conversational Booking. • Working WhatsApp-to-Database Data Flow. • Enhanced User Dashboard with Profile Management.
February: Vendor Tools and Social Features	• Develop vendor-side tools: analytics dashboard, calendar availability, and payment confirmation. • Implement social features: match-making, chat, and leaderboard. • Optimize bilingual model responses through retraining and error analysis.	• Fully Functional Vendor Dashboard. • Active Chat, Matchmaking, and Leaderboard Features. • Optimized Bilingual NLU Model.

Table 1:

Month	Work (Objectives)	Deliverables
March: Integration, Optimization, and Testing	• Integrate user and vendor interfaces with backend services. • Implement Google Sheets API for vendor booking synchronization. • Conduct unit, integration, and system testing; optimize for performance. • Perform UX testing and benchmarking.	• Fully Integrated BookForMe Platform. • Stable End-to-End Workflow (User ↔ AI Receptionist ↔ Vendor). • Detailed Test Reports and Benchmark Metrics.
April: Final Deployment and Submission	• Conduct pilot testing with selected vendors. • Deploy production-ready system on Firebase with secured integrations. • Prepare final documentation, technical report, and presentation.	• Production-Ready Deployment. • Final Project Report and Presentation Deck. • Complete Submission Package for Evaluation.

Table 2: Project Timeline and Deliverables Overview

3 Wireframes

3.1 Overview

These mockups are created using Figma and serve as a blueprint for frontend implementation. And include the user booking flow, vendor dashboard, and social features.

3.2 User Interface Mockups

3.3 Vendor Interface Mockups

3.4 Risks and Mitigation Strategies

The following table outlines the major technical, operational, and organizational risks identified during the development of the *BookForMe* system, along with corresponding mitigation strategies and recommended timelines.



Figure 2: User Homepage Wireframe: Service Discovery and Booking



Figure 3: Vendor Homepage Wireframe

Risk	Mitigation Strategy
API Rate Limits (WhatsApp / Google Sheets)	Implement exponential backoff and retry policies; throttle outbound requests; batch operations where possible; cache responses; request higher quotas in advance.
Data Consistency and Race Conditions (Concurrent Bookings)	Use Firestore transactions and idempotency keys; apply optimistic locking; implement short-lived reservation holds to avoid double-booking.
Bilingual NLU Failure (English-Urdu / Roman Urdu)	Collect diverse bilingual data; use transfer learning; continuous retraining with error logs; add fallback rule-based handlers.
Security Breaches / Key Exposure	Store credentials in Google Secret Manager; rotate keys; restrict IAM roles; enforce HTTPS and TLS; audit repository commits for sensitive data.
WhatsApp / Google API Policy Changes	Maintain modular adapter layer for APIs; track changelogs; implement fallback REST mocks for testing.
System Latency / NLU Inference Delay	Profile model inference; cache frequent queries; use async jobs for long responses; horizontally scale backend on Firebase Cloud Functions.
Cloud Cost Overrun (Firestore Reads / Functions Invocations)	Estimate cost per call; use batched reads; denormalize where appropriate; add billing alerts and caps in Firebase.
UX / Usability Shortcomings	Conduct iterative usability testing; analyze clickstream and drop-off metrics; refine prototype flows accordingly.

Table 3: Project Risks, Mitigation Strategies, and Recommended Timelines